

14 — Lei de Amdahl: Vale a Pena Otimizar Isso?

Pré-requisito: nenhum específico deste módulo — este arquivo é deliberadamente curto e independente, porque a lição que ele ensina é mais sobre **estratégia de onde gastar esforço** do que sobre mecanismo de hardware.

1. A pergunta que este arquivo responde

Você já tem, dos arquivos anteriores, ferramentas pra saber **como** otimizar uma operação (reduzir bytes movidos, aumentar AI, aumentar MLP). Falta uma pergunta anterior a todas essas: **vale a pena otimizar essa operação específica, dado o tempo total do programa?**

A Lei de Amdahl responde exatamente isso — e a resposta, com frequência, é "não, não vale, otimize outra coisa".

2. A fórmula

$$\text{Speedup_total} = 1 / ((1 - P) + P/S)$$

P = fração do tempo total ocupada pela parte que você vai otimizar

S = quanto mais rápida você consegue tornar ESSA parte especificamente

3. O exemplo que mata a intuição ingênua

Imagine um kernel de normalização (o assunto do arquivo 15) que ocupa **5%** do tempo total de um passo de treino ($P = 0.05$). Suponha que você consiga, com um trabalho de engenharia heroico, tornar esse kernel **infinitamente mais rápido** ($S \rightarrow \infty$, o limite teórico absoluto, cálculo que leva tempo zero).

$$\begin{aligned} \text{Speedup_total} &= 1 / ((1 - 0.05) + 0.05/\infty) \\ &= 1 / (0.95 + 0) \\ &= 1 / 0.95 \\ &\approx 1.053 \end{aligned}$$

O speedup total do treino inteiro é ~5.3% — mesmo tendo tornado aquele kernel **infinitamente** rápido. Os outros 95% do tempo (tudo que você não tocou) continuam exatamente do mesmo tamanho, e é isso que domina o resultado final.

PERGUNTA RÁPIDA 1: se, em vez de infinito, você conseguisse só **2x** de speedup nesse mesmo kernel de 5%, qual seria o speedup total?

► Resposta

4. A consequência prática, direto pro seu objetivo

Isso é o argumento matemático formal por trás de uma regra que você já ouviu informalmente hoje: **profile first, optimize what matters**. Antes de investir semanas otimizando um kernel específico (RMSNorm, por exemplo), a pergunta que precede qualquer trabalho de otimização é: **"que fração do tempo total esse kernel realmente ocupa?"** — porque um kernel que ocupa 40% do tempo, otimizado por 2x, vale muito mais que um kernel de 5% otimizado por 10x.

```
Kernel A: P = 0.40, S = 2    -> Speedup_total = 1 / (0.60 + 0.20) = 1.25    (25% de
ganho)
Kernel B: P = 0.05, S = 10  -> Speedup_total = 1 / (0.95 + 0.005) = 1.047 (4.7% de
ganho)
```

Otimizar o Kernel A (mesmo com um ganho local mais modesto, 2x) rende **mais que 5x o resultado final** comparado a otimizar o Kernel B com um ganho local muito mais impressionante (10x). Isso não é intuitivo até você fazer a conta — e é exatamente por isso que perfilar **antes** de otimizar não é burocracia, é a diferença entre gastar semanas de esforço em algo que renderá 4.7% ou 25%.

5. Por que isso é literalmente o primeiro passo do fluxo de trabalho do seu lab

Isso conecta diretamente com dois lugares do seu próprio repositório: o **KERNEL_WORKFLOW.md** (que exige medir antes de otimizar) e a **Phase 12** do seu **ROADMAP.md** ("Transformer Integration"), que existe especificamente porque *"um resultado de microbenchmark não é um resultado de treino"* — o próprio roadmap já assume a lógica de Amdahl, mesmo sem citar a fórmula: otimizar um kernel isolado só importa na medida em que aquele kernel representa uma fração relevante do tempo de treino real. É por isso que a Phase 12 mede explicitamente "a fração do tempo de step ocupada pela normalização" **antes** de comemorar qualquer ganho de kernel isolado.

6. Por que isso importa para CUDA (e pro seu North Star especificamente)

O North Star do seu lab é sobre otimizar kernels de normalização — mas normalização tipicamente ocupa uma fração **pequena** do tempo total de um passo de treino de Transformer (a maior parte do tempo geralmente vai pra matmuls de atenção e das camadas lineares). Isso não invalida o projeto — mas significa que uma parte real do trabalho de pesquisa, antes mesmo de otimizar qualquer kernel, é **medir exatamente qual P** a normalização realmente ocupa no seu caso de teste, porque é esse número, junto com o **S** que você conseguir, que determina se o resultado final é interessante o suficiente pra virar um capítulo de tese ou só uma nota de rodapé. Isso é literalmente o "milestone 5" (Transformer Integration) do seu **ROADMAP.md**.

7. Erros comuns

- Otimizar o que é mais **interessante** de otimizar, em vez do que ocupa mais tempo — Amdahl existe justamente pra corrigir esse viés.
- Esquecer que **P** muda: se você otimiza uma parte do sistema, a fração de tempo ocupada pelas outras partes **aumenta** relativamente (o denominador do "tempo total" ficou menor) — a próxima rodada de otimização pode precisar mirar em outro lugar.
- Achar que Amdahl diz "não vale a pena otimizar nada pequeno" — não é isso. Diz "otimizar algo pequeno tem **teto de retorno** baixo, então pondere esforço contra esse teto antes de começar", não

que seja proibido.

PREDICT BEFORE RUNNING (cálculo)

Um passo de treino tem: 60% do tempo em matmuls, 25% em atenção, 10% em normalização, 5% em outras operações (ativações, etc.).

1. Se você conseguir otimizar normalização em 3x, qual o speedup total do passo de treino?
2. Se você conseguir otimizar matmuls em apenas 1.2x (20% mais rápido), qual o speedup total?
3. Compare os dois resultados. O que isso te diz sobre onde vale mais a pena investir esforço de engenharia, mesmo que "otimizar matmul em 20%" pareça um objetivo menos ambicioso que "otimizar normalização em 3x"?

► Explicação